# Multi-stage code generation: Using different LLM's at different stages

**Damir Numić-Meša, Matej Zubić and Toni Borina**

[1] University of Zagreb Faculty of Electrical Engineering and Computing, Croatia

**Abstract**

**Originality/Value –** Step based architecture for solving geospatial problems
**Keywords –** Machine Learning; Deep Learning; LLM; Prompt Engineering; Geospatial Analysis; Autonomous GIS; LUCAS dataset;
**Paper Type –** Research paper.

## 1. Introduction

In today's world, where computers play an essential role in our daily lives and AI has reached incredible heights of innovation, we are constantly exploring new ways to apply emerging technologies. However, one area where AI has not yet made a significant impact in reaching a wider audience is programming languages.

This paper introduces a multistep based architecture designed to simplify geospatial analysis using open-source large language models (LLMs). These models have transformed how humans interact with computers by advancing natural language processing (NLP). LLMs are effective because of their advanced design and the large datasets they are trained on. While many LLMs are expensive and proprietary, our focus is on using free, open-source models to make these tools more accessible.

A key element of this study is the LUCAS dataset, which stands for Land Use/Cover Area Frame Statistical Survey. LUCAS provides detailed information on soil properties, land use, and land cover across Europe. This dataset is critical for understanding soil health, land degradation, and environmental changes, making it an important resource for scientists and policymakers.

Our project uses the LUCAS dataset to address common challenges in geospatial analysis, such as working with complex spatial data, combining different data sources, and visualizing information. By using open-source LLMs, we aim to create an easy-to-use tool that makes geospatial analysis more accessible to everyone.

By allowing users to input natural language queries, our assistant simplifies geospatial and statistical analysis, eliminating the need for advanced technical skills. This paper evaluates how well open-source LLMs perform in understanding queries, analyzing geospatial data, and producing accurate results. Ultimately, our goal is to make geospatial analysis tools more user-friendly and accessible, using open-source LLMs to bridge the gap between complex data and everyday users.

## 2. Related work

Early efforts such as GeoGPT (Zhang, Wei, Wu, He, & Yu, 2023), MapGPT (Fernandez & Dube, 2023), and GeoLLM (Manvi, et al., 2024) have explored the potential of AI in spatial

data analysis, solution generation, and decision-making. Recent advancements in geospatial AI include applications like flood depth estimation (Akinboyewa, Ning, Lessani, & Li, 2024), flood severity mapping (Feng, Brenner, & Sester, 2020), disaster data interpretation (Hu, et al., 2023), place identity analysis (Jang, et al., 2023), and ecological studies (Han, et al., 2023). While large foundation models outperform task-specific ones in certain text-based geospatial tasks (Mai, et al., 2023), they still struggle with multimodal data that combines diverse formats.

Our research focuses on leveraging open-source LLMs for geospatial and soil science, distinguishing it from many studies that rely on proprietary models like ChatGPT. We investigate how AI assistants can effectively communicate with users, emphasizing the role of prompt engineering in generating precise and actionable instructions. Additionally, we explore the structured implementation of such agents to advance autonomous GIS systems, where AI automates spatial data processing and visualization.

A recent study introduced the concept of autonomous GIS, where AI-driven systems independently handle spatial problems (Li & Ning, 2023). A prototype, LLM-Geo (Li & Ning, 2023), used GPT-4 to generate numerical summaries, graphs, and maps, significantly reducing manual workload. However, it lacks key features like logging and code validation, which are crucial for real-world deployment. While straightforward queries can be solved directly by models like GPT-4o, complex spatial tasks would benefit from a structured approach where the LLM first formulates a plan before execution. This strategy could lead to more accurate, reliable, and interpretable results.

## 3. Methodology

This research focuses on utilizing publicly available pretrained large language models (LLMs) hosted on the Hugging Face platform. The primary aim is to design an intelligent assistant tailored to simplify statistical analysis. The assistant is specifically intended for individuals who understand statistical concepts but lack the programming skills necessary to implement them. By bridging this gap, the assistant enables users to carry out complex statistical queries and analysis through natural language input, eliminating the need for coding knowledge.

The system is designed to help the user find, calculate or visualize specific data from the LUCAS dataset. Based on a user query, it will perform multiple steps to find the best way to execute the python code and present the user with the final solution. The final solution can be in the form of numbers, graph or text.

### 3.1 Dataset

In this research we used LUCAS (Land Use and Coverage Area frame Survey) Soil dataset from 2018. It is a comprehensive version of the LUCAS survey, providing high-quality data on land use, land cover, and soil properties across the European Union. The dataset provides detailed information about the land use and land cover, soil properties such as texture, organic carbon, pH levels and others. Additionally, it contains subsoil properties such as organic carbon content and calcium carbonate content at bigger depths (20-30cm) and geospatial and administrative data.

### 3.2 Architecture

On Figure 1 system architecture is displayed. First, we receive the user query, that query is then embedded in the system prompt for data transformation. Together with the user query,

system prompt also contains the description of the columns with its types, and we also add the list of the available pandas functions we suggest it to use. Descriptions of the columns are taken from the official documentation of the dataset. This ensures that the Coder LLM has a complete understanding of the dataset's structure and semantics, reducing ambiguity and increasing the quality of the generated instructions. Additionally, we instruct the Coder LLM only to return the python code without additional text, tell it where the data about the map of Europe is stored and request it to put the final transformation result in specified variable. The preloaded information about where the map of Europe is stored, along with instructions to place the final transformed dataset into a specific variable, reduces complexity and the risk of error.

From the response of the Coder LLM we extract the python code and clean it by removing all the comments, import statements, print statements and the empty lines. This cleaning step ensures that the code is streamlined and free of unnecessary clutter, making it more reliable and efficient for execution. After cleaning the code, we execute it and retrieve the description of the data by calling pandas.Dataframe.info() function. This intermediate validation step guarantees that the visualization LLM receives accurate information about the transformed dataset, avoiding potential mismatches between transformation and visualization phases.

In parallel we can also execute the call to Instruct LLM which will retrieve the type of visualization, and the response will be used as input to the visualization Coder LLM. We need this part of the pipeline since we noticed that Coder LLM we use for visualization is sometimes struggling to conclude how the data should be presented to the user. Doing one additional call to the Instruct LLM significantly reduces the errors we experienced while having Coder LLM decide how to present the data. In system query tell the LLM to return only one of the following:

- **print**: for simple transformations whose results should be directly printed
- **graph**: for queries that require visualizing transformed data as plots or graphs (e.g., histograms, scatter plots)
- **map**: for queries requiring overlaying transformed results on a map of Europe already loaded as a GeoDataFrame (europe_gdf)

For the visualization part of the task, we have different system queries depending on the type of visualization we determine in the previous step. We enrich each of these system queries with user query, description of dataset, and additional information about what should be considered regarding the type of visualization. This ensures that the visualization LLM has all the context it needs to generate accurate and effective visualization code. By embedding detailed metadata at every step, the process minimizes user effort and reduces the likelihood of errors. This is the last request of our pipeline, and it is sent to Coder LLM.

Once we receive the response, we clean the code in the same way as we cleaned the code generated for transformation, stack those two codes together and run it to retrieve the result.

As shown in the Figure 1, each request to the LLM is followed by a validation step. The validation step after the Coder LLMs is trying to execute the code retrieved from the LLM. In case the validation is successful the current step is finished and the next one in the pipeline may start. If validation fails, we restart the pipeline from the current step but this time with a different system query. With the standard query parameters for the step, this query also contains the code from the previous response together with the exception the code raised

during its execution. Additionally, it is possible to specify different LLM for the retry steps. If validation fails after the call to the Instruct LLM, we simply retry the same request.

This approach enhances reliability by validating each step before moving forward, preventing errors from propagating through the pipeline. By incorporating exception feedback and allowing retries with an adjusted system query or even a different LLM, the pipeline a has higher chance of generating correct and executable code.

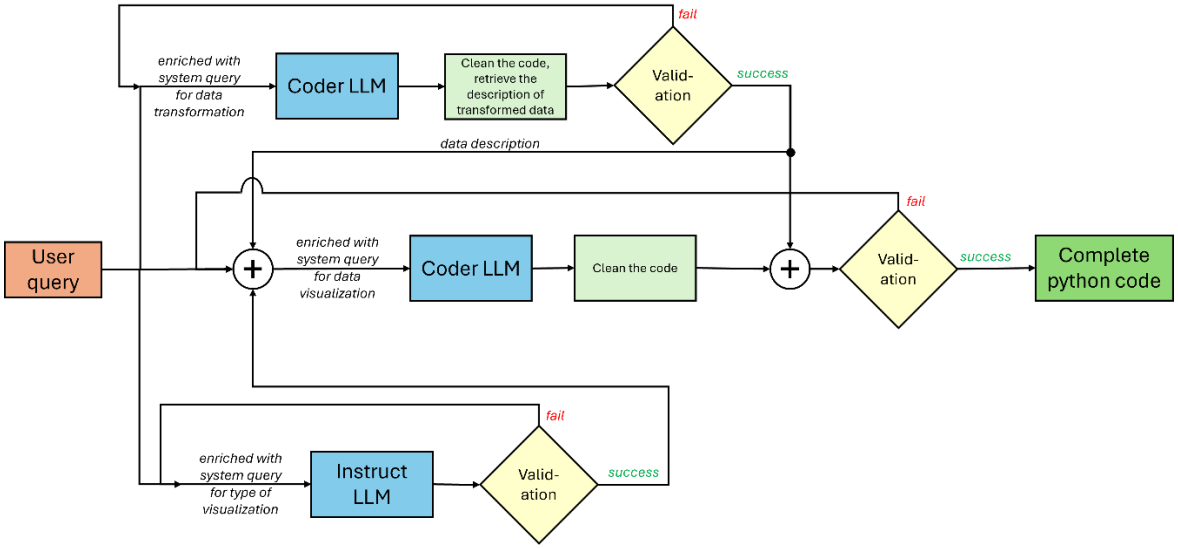



Figure 1 System architecture

The proposed approach is effective because it breaks the process into clear, modular steps, ensuring clarity and reducing complexity. Embedding detailed metadata and dataset descriptions at each stage enables the LLMs to generate accurate and context-aware outputs. Intermediate validation, such as inspecting transformed data, ensures alignment between transformation and visualization. By guiding the Coder LLM with curated functions and focusing it solely on code generation, the pipeline produces efficient and reliable code. Additionally, the structured design enhances flexibility, scalability, and reproducibility, while streamlining the user experience for consistent and high-quality results. It also gives an opportunity to validate each stage of the code generation independently. We can also implement some additional retry mechanisms that will retry the query to the Coder LLM if the current solution does not satisfy some requirements.

# 1 Results

To evaluate generated code we used multiple metrics - METEOR, BLEU, cosine similarity, AST, Levenstein distance and ROUGE. We also decided to include additional custom based metric which utilizes LLM-s for comparison of generated and reference code, and outputs similarity value in range [0,1] with additional reasoning for given output. We provide results for geographical queries in the table below. For each query we provide generated code. If the query was successful in multiple successive runs we additionally provide code output as image.

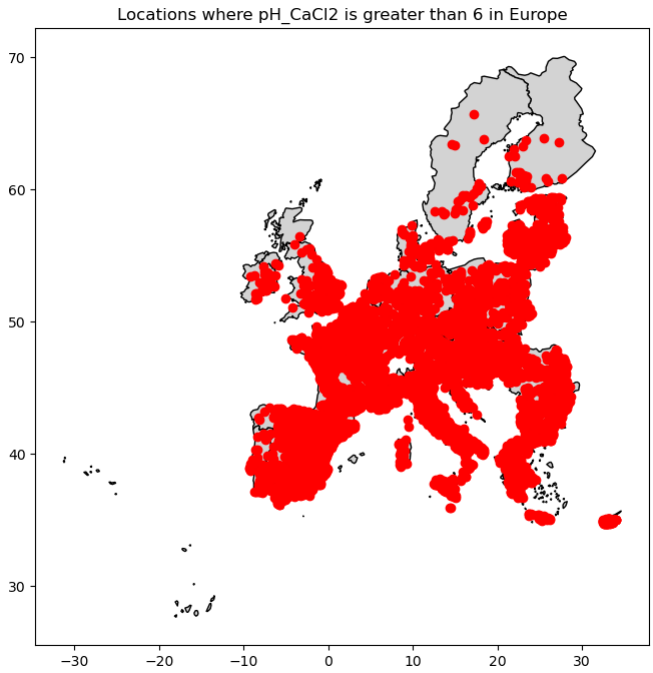
| ID | METEOR | BLEU | Cosine Similarity | AST | Levenstein Distance | ROUGE-1 | ROUGE-2 | ROUGE-L | LLM |
|---|---|---|---|---|---|---|---|---|---|
| 1. | 0.4312 | 0.0757 | 0.4635 | 0 | 15 | 0.3313 | 0.1491 | 0.3067 | 0.7 |
| 2. | 0.4046 | 0.1308 | 0.4931 | 0 | 11 | 0.3311 | 0.1745 | 0.3179 | 0.7 |
| 3. | 0.4274 | 0.1441 | 0.45 | 0 | 21 | 0.3256 | 0.1882 | 0.3256 | explanation only |
| 4. | 0.399 | 0.0836 | 0.4851 | 0 | 21 | 0.4574 | 0.2473 | 0.383 | 0.85 |
| 5. | 0.485 | 0.0697 | 0.4307 | 0 | 17 | 0.3724 | 0.1538 | 0.3448 | explanation only |
| 6. | 0.3865 | 0.0679 | 0.5311 | 0 | 17 | 0.4667 | 0.2568 | 0.3733 | 0.85 |
| 7. | 0.3907 | 0.0488 | 0.4534 | 0 | 21 | 0.4049 | 0.1863 | 0.3558 | 0.5 |
| 8. | 0.3041 | 0.0385 | 0.4733 | 0 | 33 | 0.4076 | 0.1627 | 0.3507 | 0.7 |
| 9. | 0.4179 | 0.0531 | 0.4111 | 0 | 13 | 0.3415 | 0.1379 | 0.2341 | 0.6 |
| 10. | 0.5406 | 0.1091 | 0.4746 | 0 | 25 | 0.3679 | 0.1905 | 0.283 | 0.8 |

Table 1: Results for geographical user queries

1. Can you plot the locations of all points where the pH_CaCl2 is greater than 6 on a map, and save the image as a PNG file?

data > generated > Llama 3.1 8B Instruct 128k > geo > 1.txt

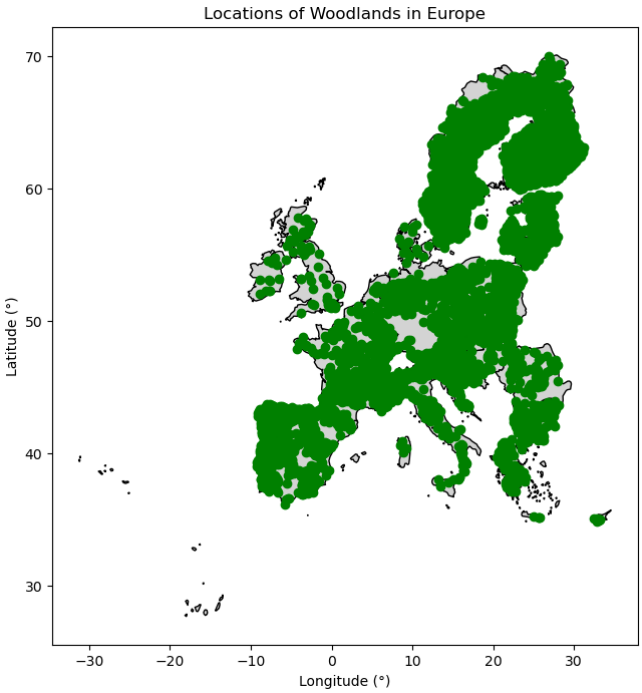
```python
def solve(data_gdf: gpd.GeoDataFrame, europe_gdf: gpd.GeoDataFrame):
    transformed_gdf = data_gdf.copy()
    # Filter points where pH_CaCl2 is greater than 6
    filtered_points = transformed_gdf[transformed_gdf['pH_CaCl2'] > 6]
    # Save the filtered GeoDataFrame to a new variable for plotting later if needed
    plotting_data = filtered_points
    # Store the result in 'transformed_gdf'
    transformed_gdf = filtered_points.copy()
    fig, ax = plt.subplots(figsize=(10, 8))
    europe_gdf.plot(ax=ax, edgecolor='black', color='lightgray')
    transformed_gdf.plot(ax=ax, column=None, markersize=1)
    ax.set_title('Locations of points where pH_CaCl2 > 6 on a map')
    plt.savefig('pH_map.png')
```




2. Can you plot the locations where LC0_Desc=Woodland on the map, and save the it as a PNG file?

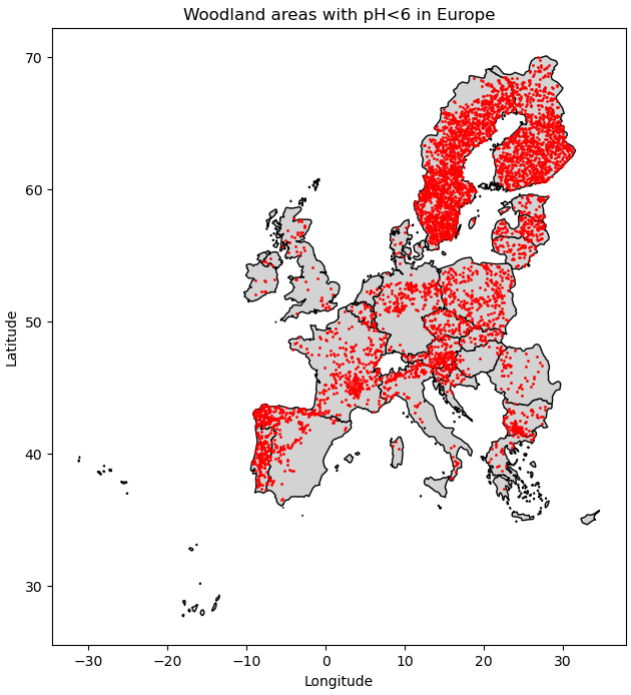
data > generated > Llama 3.1 8B Instruct 128k > geo > 2.txt

```python
def solve(data_gdf: gpd.GeoDataFrame, europe_gdf: gpd.GeoDataFrame):
    transformed_gdf = data_gdf.copy()
    filtered_data = transformed_gdf[transformed_gdf['LC0_Desc'] == 'Woodland']
    fig, ax = plt.subplots(figsize=(10, 8))
    europe_gdf.plot(ax=ax, edgecolor='black', color='lightgray')
    transformed_gdf[transformed_gdf['LC0_Desc'] == 'Woodland'].plot(ax=ax)
    ax.set_title('Locations of Woodland on a map')
    plt.savefig('woodland_map.png')
```




3. Plot all the points where LC0_Desc=Woodland and pH_CaCl2<6 on the map, and save the result as a png.

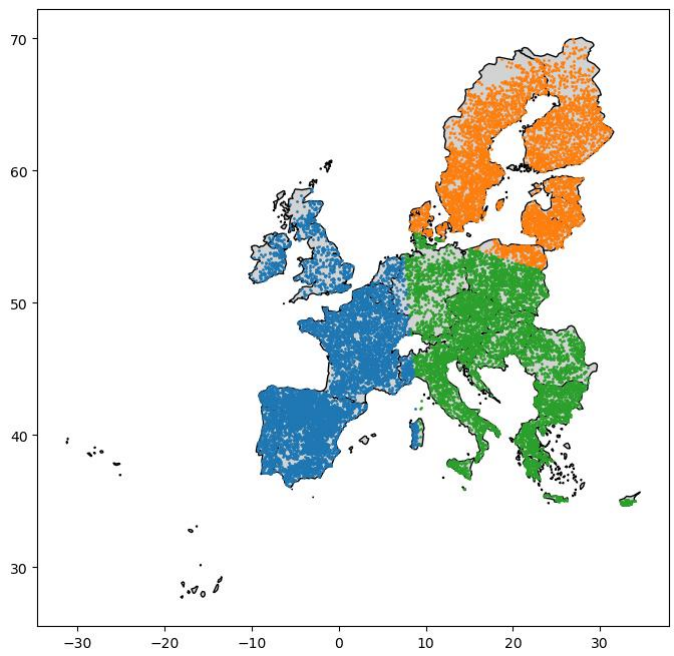
```
data > generated > Llama 3.1 8B Instruct 128k > geo > ≡ 3.txt
1  def solve(data_gdf: gpd.GeoDataFrame, europe_gdf: gpd.GeoDataFrame):
2      transformed_gdf = data_gdf.copy()
3      filtered_data = transformed_gdf[(transformed_gdf['LC0_Desc'] == 'Woodland') & (transformed_gdf['pH_CaCl2'] < 6)]
4      fig, ax = plt.subplots(figsize=(10, 8))
5      europe_gdf.plot(ax=ax, edgecolor='black', color='lightgray')
6      transformed_gdf[(transformed_gdf['LC0_Desc'] == 'Woodland') & (transformed_gdf['pH_CaCl2'] < 6)].plot(ax=ax)
7      ax.set_title('Locations of Woodland with pH<6 on a map')
8      plt.savefig('woodland_low_pH_map.png')
```



4. Can you use KMeans clustering to group geographic points based on latitude and longitude into three clusters, plot them on a map, and save the result as a PNG file?

data > generated > Llama 3.1 8B Instruct 128k > geo > 4.txt

```python
def solve(data_gdf: gpd.GeoDataFrame, europe_gdf: gpd.GeoDataFrame):
    from sklearn.cluster import KMeans
    transformed_gdf = data_gdf.copy()
    kmeans = KMeans(n_clusters=3)
    kmeans.fit(data_gdf[['TH_LAT', 'TH_LONG']])
    data_gdf['cluster'] = kmeans.predict(data_gdf[['TH_LAT', 'TH_LONG']])
    fig, ax = plt.subplots(figsize=(10, 8))
    europe_gdf.plot(ax=ax, edgecolor='black', color='lightgray')
    transformed_gdf['cluster'] = KMeans(n_clusters=3).fit_predict(transformed_gdf[['TH_LAT', 'TH_LONG']])
    ax.scatter(transformed_gdf['TH_LAT'], transformed_gdf['TH_LONG'], c=transformed_gdf['cluster'])
    plt.savefig('kmeans_map.png')
```




5. Can you create a map showing locations where potassium (K) levels are above the median, and save the map as a PNG file?

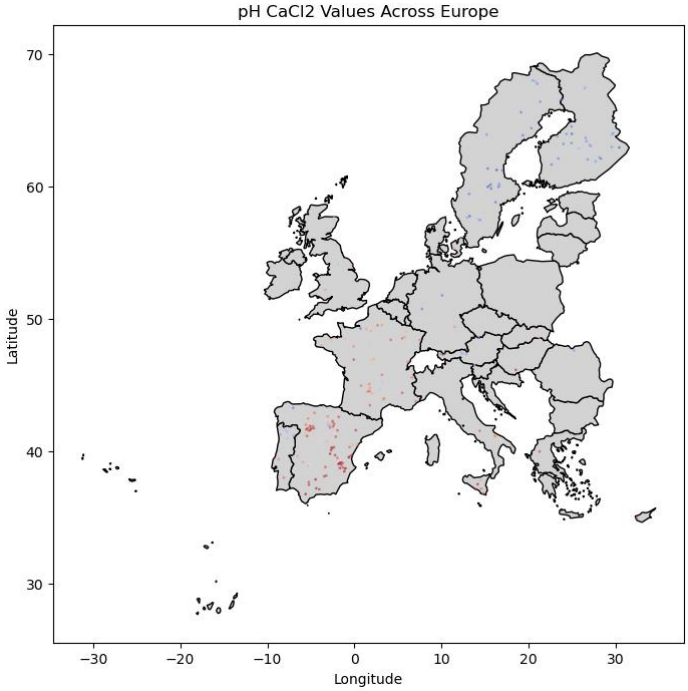
data > generated > Llama 3.1 8B Instruct 128k > geo > 5.txt

```python
def solve(data_gdf: gpd.GeoDataFrame, europe_gdf: gpd.GeoDataFrame):
    transformed_gdf = data_gdf.copy()
    data_gdf['K_above_median'] = (data_gdf['K'] > stats.scoreatpercentile(data_gdf['K'], 50))
    fig, ax = plt.subplots(figsize=(10, 8))
    europe_gdf.plot(ax=ax, edgecolor='black', color='lightgray')
    transformed_gdf[transformed_gdf['K_above_median'] == True].plot(ax=ax, marker='o', markersize=1)
    plt.savefig('k_potassium_map.png')
```

6. Can you create a map showing pH_CaCl2 values across Europe? I'd like the dots to be colored by pH_CaCl2 values, and save it as an image.

data > generated > Llama 3.1 8B Instruct 128k > geo > 6.txt

```
def solve(data_gdf: gpd.GeoDataFrame, europe_gdf: gpd.GeoDataFrame):
    transformed_gdf = data_gdf.copy()
    data_gdf['pH_CaCl2_color'] = pd.cut(data_gdf['pH_CaCl2'], bins=5, labels=['low', 'medium_low', 'median', 'medium_high', 'high'])
    fig, ax = plt.subplots(figsize=(10, 8))
    europe_gdf.plot(ax=ax, edgecolor='black', color='lightgray')
    transformed_gdf.plot(ax=ax, column='pH_CaCl2', legend=True)
    plt.savefig('ph_map.png')
```




7. Can you show me a map of Europe marking only the locations with the highest potassium ('K') levels? Just highlight the top 10% of spots and save it as an image.

data > generated > Llama 3.1 8B Instruct 128k > geo > 7.txt

```
def solve(data_gdf: gpd.GeoDataFrame, europe_gdf: gpd.GeoDataFrame):
    transformed_gdf = data_gdf.copy()
    data_gdf['K_above_median'] = (data_gdf['K'] > stats.scoreatpercentile(data_gdf['K'], 90))
    fig, ax = plt.subplots(figsize=(10, 8))
    europe_gdf.plot(ax=ax, edgecolor='black', color='lightgray')
    transformed_gdf[transformed_gdf['K_above_median'] == True].plot(ax=ax, marker='o', markersize=1)
    plt.savefig('high_potassium_map.png')
```

8. Could you make a map showing where soil pH_H2O levels in Europe are acidic versus basic? Mark acidic spots in red and basic spots in blue, and save it as an image.

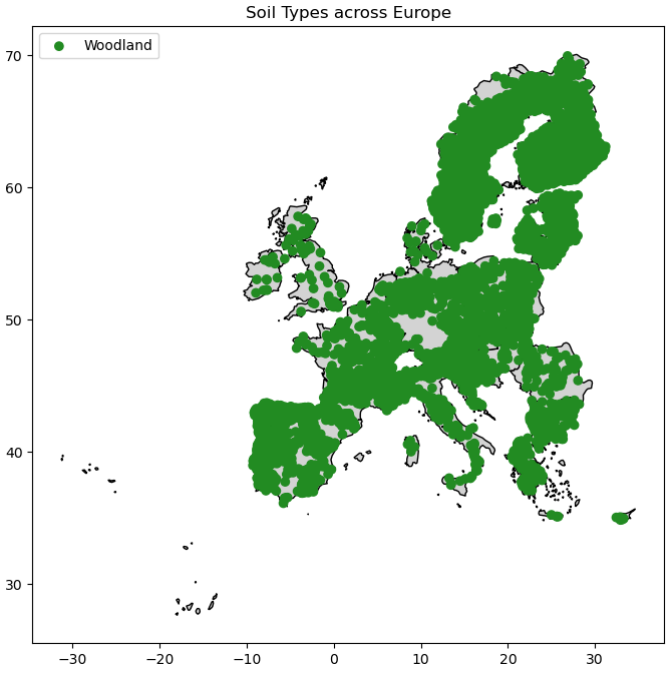
data > generated > Llama 3.1 8B Instruct 128k > geo > 8.txt

```python
def solve(data_gdf: gpd.GeoDataFrame, europe_gdf: gpd.GeoDataFrame):
    transformed_gdf = data_gdf.copy()
    data_gdf['pH_H2O_category'] = pd.cut(data_gdf['pH_H2O'], bins=[0, 7, 14], labels=['acidic', 'basic'])
    fig, ax = plt.subplots(figsize=(10, 8))
    europe_gdf.plot(ax=ax, edgecolor='black', color='lightgray')
    transformed_gdf['pH_H2O_category'] = transformed_gdf['pH_H2O'].apply(lambda x: 'acidic' if x < 7 else 'basic')
    transformed_gdf.plot(ax=ax, column='pH_H2O_category', legend=True, markersize=1)
    plt.savefig('ph_map.png')
```

9. Can you create a colorful map showing different types of soil by 'LC0_Desc' across Europe? I'd like each soil type marked in a different color and saved as an image.

data > generated > Llama 3.1 8B Instruct 128k > geo > 9.txt

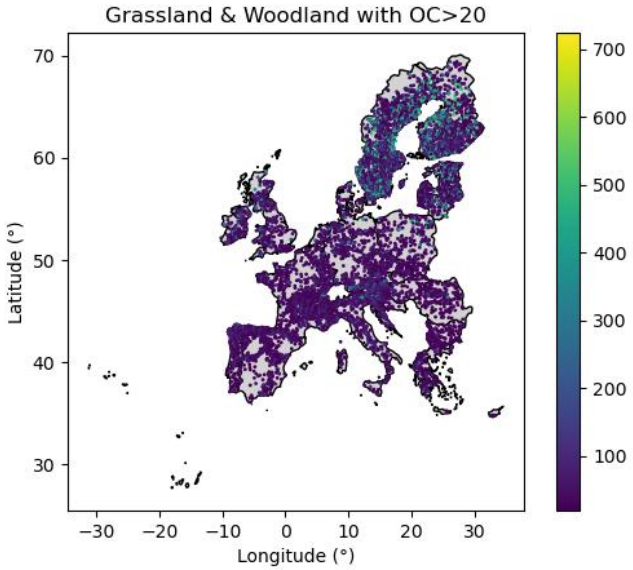
```python
def solve(data_gdf: gpd.GeoDataFrame, europe_gdf: gpd.GeoDataFrame):
    # Import necessary libraries and create a copy of the original GeoDataFrame to avoid modifying it directly
    import pandas as pd
    transformed_gdf = data_gdf.copy()
    # Filter for LC0_Desc column values that are not 'Water'
    transformed_gdf = transformed_gdf[~transformed_gdf['LC0_Desc'].isin(['Water'])]
    # Group by LC0_Desc and create a new GeoDataFrame with unique colors for each soil type
    unique_soil_types = transformed_gdf.groupby('LC0_Desc')['geometry']
    colors = ['red', 'green', 'blue', 'yellow', 'purple', 'orange', 'pink']
    transformed_gdf = pd.concat([pd.DataFrame({'color': [c] * len(unique_soil_types.get_group(soil_type)),
                                               'geometry': unique_soil_types.get_group(soil_type)}) for soil_type, c in zip(unique_s
    import matplotlib.pyplot as plt
    # Create a figure and axis object to plot the map of Europe with 'transformed_gdf' data overlayed on it.
    fig, ax = plt.subplots(figsize=(10, 8))
    # Plot the base map of Europe using europe_gdf
    europe_gdf.plot(ax=ax, edgecolor='black', color='lightgray')
    # Overlay the transformed data from 'transformed_gdf'
    # Since we have a column named 'color' in our GeoDataFrame which contains different colors for each soil type,
    # We will use this to style the markers on the map.
    for i, row in transformed_gdf.iterrows():
        ax.plot(row.geometry.x, row.geometry.y, marker='o', color=row.color, markersize=1)
    # Set a title for the plot and show it.
    plt.title('Soil Types in Europe')
    plt.show()
```




10. Plot all the LC0_Desc='Grassland' and LC0_Desc='Woodland' points where 'OC'>20 on the map, and save the map as a png.

data > generated > Llama 3.1 8B Instruct 128k > geo > 10.txt

```
def solve(data_gdf: gpd.GeoDataFrame, europe_gdf: gpd.GeoDataFrame):
    # Filter the data for LC0_Desc='Grassland' and LC0_Desc='Woodland'
    grassland_woodland_gdf = data_gdf[(data_gdf['LC0_Desc'] == 'Grassland') | (data_gdf['LC0_Desc'] == 'Woodland')]
    # Filter grassland_woodland_gdf for OC > 20
    filtered_data = grassland_woodland_gdf[grassland_woodland_gdf['OC'] > 20]
    transformed_gdf = filtered_data.copy()
    import matplotlib.pyplot as plt
    # Create a figure and axis object for plotting
    fig, ax = plt.subplots(figsize=(10, 8))
    # Set the base map using europe_gdf
    europe_gdf.plot(ax=ax, edgecolor='black', color='lightgray')
    # Filter data from transformed_gdf based on LC0_Desc and OC conditions
    filtered_data = transformed_gdf[(transformed_gdf['LC0_Desc'] == 'Grassland') | (transformed_gdf['LC0_Desc'] == 'Woodland')]
    filtered_data = filtered_data[filtered_data['OC'] > 20]
    # Plot the filtered data on top of the map as markers
    filtered_data.plot(ax=ax, column='LC0_Desc', legend=True, markersize=10)
    # Set title and labels for better visualization
    ax.set_title('Grassland and Woodland with OC>20')
    ax.set_xlabel('Longitude')
    ax.set_ylabel('Latitude')
    # Save the plot to a file as specified in the query context
    plt.savefig('grassland_woodland_oc_map.png', dpi=300)
    # Show the plot for verification purposes (optional)
    plt.show()
```




## 2 Discussion

## 3 Conclusions

This study explored the use of small open-source large language models (LLMs) for geospatial analysis. By designing a modular pipeline that incorporates different LLMs for

data transformation and visualization, we demonstrated that AI-powered assistants can effectively interpret user queries and generate relevant geospatial insights. The approach significantly reduces the technical expertise required to analyze geospatial data, making it accessible to a broader audience. The evaluation metrics indicate that our method generates reliable and accurate code for a variety of spatial queries.

## 4 Theoretical implications

This research contributes to the growing field of AI-driven geospatial analysis by demonstrating how multiple LLMs can be orchestrated in a structured manner. Our findings highlight the importance of system prompts, data structure descriptions, and validation steps in improving the reliability of AI-generated code. Moreover, our approach supports the broader theoretical framework of autonomous GIS by showcasing how AI agents can systematically execute spatial tasks with minimal human intervention.

## 5 Practical implications

The practical applications of this study are significant, especially for users who lack programming skills but require insights from geospatial datasets. By allowing users to query geospatial data in natural language, our approach facilitates data-driven decision-making in fields such as environmental science, agriculture, and urban planning. Furthermore, our pipeline can be adapted to various LLM architectures, making it a flexible and scalable solution for real-world applications.

## 6 Limitations and future research

Despite its effectiveness, our approach has certain limitations. First, while our validation steps improve reliability, they do not eliminate errors in code generation, or errors that might stem from phase code-joining. Future research could explore more advanced error-handling mechanisms, such as reinforcement learning techniques to refine model outputs. Second, our system currently focuses on geospatial queries related to the LUCAS dataset. Expanding support for additional datasets and integrating multimodal AI models could enhance the system’s applicability.

## References


Akinboyewa, T., Ning, H., Lessani, M. N., & Li, Z. (2024). Automated Floodwater Depth Estimation Using Large Multimodal Model for Rapid Flood Mapping.

Feng, Y., Brenner, C., & Sester, M. (2020). Flood severity mapping from Volunteered Geographic Information by interpreting water level from images containing people: A case study of Hurricane Harvey. *ISPRS Journal of Photogrammetry and Remote Sensing*.

Fernandez, A., & Dube, S. (2023). Core Building Blocks: Next Gen Geo Spatial GPT Application.

Han, B. A., Varshney, K. R., LaDeau, S., Subramaniam, A., Weathers, K. C., & Zwart, J. (2023). A synergistic future for AI and ecology.

Hu, Y., Mai, G., Cundy, C., Choi, K., Lao, N., Liu, W., & et al. (2023). Geo-knowledge-guided GPT models improve the extraction of location descriptions from disaster-related social media messages. *International Journal of Geographical Information Science*.

Jang, K. M., Chen, J., Kang, Y., Kim, J., Lee, J., & Duarte, F. (2023). Understanding Place Identity with Generative AI.

Li, Z., & Ning, H. (2023). Autonomous GIS: the next-generation AI-powered GIS. *International Journal of Digital Earth*.

Mai, G., Huang, W., Sun, J., Song, S., Mishra, D., Liu, N., & et al. (2023). On the Opportunities and Challenges of Foundation Models for Geospatial Artificial Intelligence.

Manvi, R., Khanna, S., Mai, G., Burke, M., Lobell, D., & Ermon, S. (2024). GeoLLM: Extracting Geospatial Knowledge from Large Language Models.

Zhang, Y., Wei, C., Wu, S., He, Z., & Yu, W. (2023). GeoGPT: Understanding and Processing Geospatial Tasks through An Autonomous GPT.